Fast Gaussian Samples with Hadamard Transforms

Zachariah Schwab

January 2026

Abstract

Gaussian variates are a core primitive in Monte Carlo rendering, stochastic differential equation solvers, and Bayesian inference. Most software systems obtain them by transforming uniform pseudorandom numbers using scalar or SIMD-friendly methods such as Box–Muller, the polar method, or Ziggurat, which typically cost on the order of 5–10 cycles per single-precision sample on modern CPUs. An alternative approach is to maintain a *pool* of Gaussian values and repeatedly apply randomized (approximately) orthogonal maps to the pool, amortizing the cost of introducing fresh randomness across many consumed variates. This pool-based paradigm targets workloads that naturally consume vector-valued Gaussians (e.g. batches of Wiener increments) rather than producing independent scalar draws.

This work implements and evaluates an AVX-512 pool-based Gaussian mixer on x86. The construction maintains a 256-dimensional pool in registers, applies a 256-point Walsh–Hadamard transform followed by scaling by $1/16$ (yielding an orthogonal map in exact arithmetic), and injects randomness via diagonal sign matrices whose entries are derived from a vectorized xoshiro128++ state. In exact arithmetic the deterministic transform is an involution; in single precision, applying it twice yields a maximum absolute error of 7.2×10^{-7} and an RMS error of 1.1×10^{-7} over 10^5 random inputs. Lagged autocorrelation tests over 50,000 steps in dimension 256 show no temporal or cross-coordinate correlations beyond what is expected from finite-sample variability (noting that the mixer is a correlated pool update, not an i.i.d. scalar generator). On a Ryzen 7 7800X3D, the kernel updates the full 256-dimensional pool in ≈ 43.5 ns (about 0.17 ns per pool entry updated, i.e. ≈ 0.85 cycles per entry at 5.0 GHz). These results suggest that, when a pool abstraction is appropriate, maintaining vector-valued Gaussians via a Hadamard-based mixer can be made an effectively negligible incremental cost on contemporary CPUs.

1 Introduction

Many simulation and rendering workloads require large numbers of independent or weakly dependent Gaussian random variables: Monte Carlo path tracing, stochastic differential equation solvers, and Bayesian inference all use Gaussian noise as a basic primitive. In most software systems, these samples are obtained by transforming a stream of uniform pseudorandom numbers via a scalar or SIMD-friendly normal generator such as Box–Muller, the polar method, or Ziggurat [4, 13, 14, 20]. On contemporary CPUs, well-engineered *ulp-accurate* implementations of these methods typically cost on the order of 5–10 cycles per single-precision sample [23, 8], with a nontrivial fraction of that budget spent on transcendental functions or rejection sampling.

A complementary line of work accelerates Gaussian generation by approximating the inverse Gaussian CDF using piecewise schemes with small lookup tables, trading ulp-level accuracy for speed. Sheridan-Methven and Giles [17] benchmark optimized implementations and report that on Intel hardware a piecewise-linear inverse-CDF using vector intrinsics achieves 0.5 ± 0.1 cycles per single-precision sample, close to a measured "read and write" baseline of 0.4 ± 0.1 cycles; for comparison, Intel MKL VSL is reported at 3.4 ± 0.1 cycles (high accuracy) or 2.6 ± 0.1 cycles (low accuracy) in single precision [17]. These timings measure the inverse-CDF evaluation itself (given uniform inputs) and target Monte Carlo regimes where approximation error can be managed (e.g. via multilevel methods) rather than enforcing ulp-accurate normality. While the present work uses a different abstraction—a correlated Gaussian pool updated by orthogonal mixing rather than an i.i.d. scalar stream—the comparison is useful for calibrating absolute throughput against state-of-the-art approximate inverse-CDF generators.

A different point in the design space, explored by Wallace and Brent [21, 5, 6], is to maintain a fixed-size *pool* of Gaussian values and repeatedly apply orthogonal transforms to the pool, occasionally reseeding it from a uniform RNG. Orthogonality ensures that a pool initially distributed as $N(0, I)$ remains Gaussian, while reusing the same values in different linear combinations amortizes the cost of generating truly fresh samples. Wallace-style pool generators have been shown to achieve "a few cycles per sample" on vector machines and GPUs [12, 16]; however, I am not aware of a detailed public writeup of an AVX-512-tuned implementation on x86.

It is also helpful to keep a geometric picture in mind. A Gaussian vector

admits a radius–direction decomposition

$$Z = RU,$$

where U is uniform on the unit sphere and $R \sim \chi_n$ is independent. Any orthogonal update $x \leftarrow Qx$ preserves $\|x\|$ exactly, and thus (ignoring reseeding and radius correction) defines a Markov chain on the sphere. In high dimension, low-dimensional projections of a nearly-uniform direction are close to Gaussian after the appropriate scaling; informally, once the direction has mixed even moderately, many statistics of interest "look Gaussian." In the present work, this viewpoint is used only as intuition for why weakly-dependent pool generators can be effective: the formal invariance statement is the classical one, namely that if the pool is exactly $N(0, I)$, then orthogonal mixing preserves that distribution exactly.

In this work I implement and evaluate a simple but aggressive instance of this idea: a 256-dimensional Hadamard-based Gaussian mixer, fully unrolled into AVX-512 instructions and driven by a vectorized xoshiro128++ state. The core transform applies a 256-point Walsh–Hadamard transform (WHT) to the pool using a sequence of butterfly operations and AVX-512 shuffles, scales by $1/16$ to obtain an orthogonal map, and then multiplies by a random diagonal sign matrix whose entries are derived from xoshiro output. All payload data and sign patterns live in zmm registers for the duration of the transform; memory traffic is limited to loading and storing the 256-element pool and the xoshiro state.

The main empirical findings are:

- The deterministic Hadamard transform is numerically very close to an involution in single precision: over 10^5 random inputs in $\mathbb{R}^{256}$, applying the transform twice yields a maximum absolute deviation of 7.2×10^{-7} and an RMS error of 1.1×10^{-7}, which is at the level of one ulp for $f32$ values of order one (Section 3).

- When embedded in a simple lagged autocorrelation test over 50,000 steps and lags 1 through 10, the resulting Markov chain shows no detectable temporal or cross-coordinate correlations beyond what is expected from finite-sample variability at dimension 256; the maximum absolute entries of the empirical lag-ℓ correlation matrices lie between 10^{-2} and 2.1×10^{-2}, matching a $\sigma\sqrt{2 \log M}$ heuristic for $M \approx 256^2$ sample correlations (Section 3).

- On a Ryzen 7 7800X3D (Zen 4, AVX-512) the scaled mixer updates a 256-dimensional pool in 45.3 ns, or 0.177 ns per sample. At an effective

core frequency of 4.8 GHz this corresponds to ≈ 0.85 cycles per float (Section 4).

These measurements place the mixer in the same asymptotic regime as classical Wallace-type generators on vector hardware and GPUs, but on a single contemporary CPU core. In regimes where a Gaussian pool is a natural abstraction—for example, when many correlated Gaussians are consumed between occasional reseedings—the incremental cost of maintaining the pool via this Hadamard-based mixer is effectively negligible compared to both scalar Gaussian generation and realistic downstream computations.

2 Related Work

Gaussian random number generation has been studied for decades, with several algorithmic families that trade off statistical quality, computational cost, and suitability for specific hardware targets. This work sits in the family of pool-based generators that apply structured orthogonal transforms to a vector of Gaussian values, in the lineage of Wallace-style methods, but is tailored specifically to AVX-512 on x86.

2.1 Transform-from-uniform methods

The classical approach starts from a scalar or vector uniform RNG and applies an analytic transform to obtain normal variates. The Box–Muller transform [4] generates a pair of independent $N(0,1)$ samples from two $U(0,1)$ uniforms via a polar coordinate mapping; it is analytically simple but requires evaluation of log, $\sqrt{\cdot}$, and trigonometric functions, which is relatively expensive on scalar and SIMD hardware. Marsaglia and Bray's polar method replaces explicit trigonometric evaluation with a rejection sampler on the unit disc [13], reducing the cost somewhat but still relying on log and $\sqrt{\cdot}$ and introducing control-flow divergence.

The Ziggurat algorithm of Marsaglia and Tsang [14] covers the target distribution with layered rectangles and uses table lookups plus a small amount of rejection sampling. Ziggurat-type methods eliminate most transcendental calls on the hot path and are widely used in software libraries and hardware-oriented designs; refinements and careful implementations have been studied by Doornik and others [10]. Comprehensive comparisons of transform-from-uniform algorithms—including Box–Muller, polar, Ziggurat, ratio-of-uniforms, and related methods—are given by Thomas and Luk [20], who evaluate throughput, tail behavior, and hardware amenability.

On modern CPUs, high-quality implementations of these methods (e.g. Intel's VSL/Gaussian routines or Milo Yip's AVX Box–Muller/Ziggurat implementations [23]) typically achieve on the order of 5–10 cycles per single-precision sample, depending on the underlying uniform RNG, vector width, and transform.

2.2 Pool-based generators and the Wallace method

Wallace proposed an alternative class of Gaussian generators based on maintaining a *pool* of Gaussian values and repeatedly applying orthogonal transforms to the pool, rather than transforming uniforms on each call [21]. Let $x \in \mathbb{R}^K$ denote a pool of K Gaussian variables. The core idea is to periodically apply $x \leftarrow A_\ell x$ where A_ℓ is an orthogonal matrix, often composed from small block-orthogonal transformations and permutations. Linearity and orthogonality imply that if x is distributed as $N(0, I_K)$, then so is $A_\ell x$; the generator thus avoids repeated evaluation of transcendental functions and, in the idealized limit, requires few uniform variates per Gaussian, primarily for occasional reseeding and transform selection.

Because an orthogonal transform preserves the Euclidean norm, naive applications of this scheme artificially fix $\|x\|^2$, whereas a pool of i.i.d. normals should have $\|x\|^2 \sim \chi^2_K$. Wallace introduces explicit χ^2 corrections: the pool is scaled by a random factor derived from an approximate chi-square variate so that $\|x\|^2$ fluctuates appropriately [21].

Brent analysed and refined Wallace's generators, and implemented vectorized versions for Fujitsu vector processors [5, 7]. His work emphasizes (i) the need for sufficiently mixing transformation schemes (permutations plus local orthogonal blocks) to avoid hidden correlations, (ii) the efficiency of viewing the generator as a structured matrix–vector multiply on a pool, and (iii) the importance of period and dependence on the underlying uniform RNG. On these vector machines, Brent reports that the Wallace generator (RANN4) achieves significantly higher throughput than vectorized Box–Muller or polar methods, with only a few machine cycles per float for large pool sizes.

Wallace-style generators have also been implemented on GPUs. Howes and Thomas describe a GPU version of the Wallace method in *GPU Gems 3*, achieving more than 5×10^9 Gaussian samples per second on then-current NVIDIA hardware, substantially outperforming CPU-based Ziggurat implementations of that era [12]. Riesinger et al. revisit non-standard PRNGs for GPUs, including Wallace-type generators, and find them competitive with, or superior to, more traditional methods for high-throughput Gaussian gen-

eration on modern GPUs [16].

The construction in this note is conceptually in this family: it maintains a fixed-size Gaussian state and applies an orthogonal map given by a scaled Walsh–Hadamard transform followed by random diagonal sign matrices. Unlike Wallace's original design, it targets a fixed pool size of 256 tailored to AVX-512 and uses xoshiro128++ as the underlying stateful RNG for sign bits, with no explicit χ^2 corrections (since the goal here is to maintain an isotropic Gaussian state rather than produce a provably i.i.d. stream).

2.3 Hadamard transforms and structured random orthogonal maps

The Walsh–Hadamard transform (WHT) provides an orthogonal transform with entries ± 1 and $O(n \log n)$ complexity using only additions and subtractions [22, 11]. Its fast implementation, the FWHT, applies a sequence of "butterfly" operations that is particularly amenable to SIMD and hardware implementations, and WHTs are widely used in communications, coding, and signal processing.

In randomized numerical linear algebra and high-dimensional geometry, "randomized Hadamard transforms"—Hadamard matrices composed with random diagonal sign matrices and permutations—serve as efficient structured proxies for dense random orthogonal matrices. They form the core of fast Johnson–Lindenstrauss transforms (FJLT) and related random projection schemes [1, 2]. Multiplication by such a transform can be implemented in $O(n \log n)$ time using only additions, subtractions, and simple shuffles, while achieving near-isometric embeddings with high probability.

Hadamard transforms have also been applied directly to Gaussian generation. Thomas' Table-Hadamard GRNG is an FPGA-oriented generator that uses tables of approximate Gaussians and Hadamard transforms to mix many low-quality samples into high-quality Gaussian outputs in parallel [19]. The design is explicitly Wallace-inspired and takes advantage of the add/sub structure of the WHT for resource-efficient hardware implementation.

In the broader practitioner literature, it is common to note that applying a Hadamard transform to an array of independent (or suitably decorrelated) inputs acts as a kind of random projection whose outputs are approximately Gaussian by a central limit effect, especially when preceded by random sign flips or permutations, and cp4space gives a concrete sketch of a Hadamard-based Gaussian pool generator in this style [9].

The present work can be viewed as a particularly aggressive instantiation of this Hadamard-based philosophy: I fix dimension $n = 256$, implement the

FWHT entirely in AVX-512 zmm registers using permutation and butterfly macros, and inject randomness via diagonal sign matrices whose entries are extracted from a vectorized xoshiro128++ state. The resulting map $x \mapsto S_t T(x)$ is orthogonal (up to floating-point rounding), making both invariance analysis and implementation straightforward.

2.4 Hardware and SIMD-oriented GRNGs

There is considerable work on tailoring Gaussian generators to specific hardware. Thomas and Luk survey both software and hardware GRNGs, including FPGA implementations of Ziggurat and Wallace-type methods [20, 18]. NVIDIA's cuRAND library provides GPU-optimized normal generators based on Box–Muller and inverse CDF transforms, with reported throughputs in the 10^{10}–10^{11} samples/s range on contemporary GPUs [15]. AMD's recent work on Gaussian sampling for RDNA3-class GPUs reports Box–Muller implementations reaching $O(10^{11})$ single-precision normals/s on an RX 7900 XT [3].

On the CPU side, Intel's Vector Statistical Library (VSL) offers AVX-optimized Gaussian routines (e.g. `vsRngGaussian`) with reported single-precision costs of roughly 4–8 clocks per element for well-chosen base RNGs on recent Xeon processors [8]. Independent benchmarks of AVX Box–Muller and Ziggurat implementations (e.g. Yip's `normaldist-benchmark` [23]) show similar per-sample costs on pre-AVX-512 Intel hardware.

By contrast, I am not aware of a published AVX2/AVX-512 implementation of a Wallace- or Hadamard-style pool-based Gaussian generator on x86. Existing pool-based work targets vector machines, GPUs, or FPGAs, while x86 SIMD generators tend to be transform-from-uniform methods. The contribution here is to fill this gap: I implement a fixed-size (256-dimensional) Hadamard-plus-sign mixer entirely in AVX-512 registers, and empirically evaluate both its statistical properties and its performance relative to high-quality scalar Gaussian generators.

3 Correctness

The core of the construction is a deterministic 256-point Walsh–Hadamard transform followed by a random diagonal sign matrix driven by xoshiro128++. In exact arithmetic the deterministic transform is an involution and the full mixer is an orthogonal map on $\mathbb{R}^{256}$ for each fixed sign pattern. In this section I (i) state these properties explicitly, (ii) report internal-state checks

that the single-precision implementation behaves as an approximate orthogonal map without introducing detectable temporal or cross-coordinate correlations, and (iii) describe scalar test batteries (TestU01 and PractRand) applied to a stream derived from the mixer via the Gaussian CDF.

The experiments below separate two regimes. When $x_0 \sim N(0, I_{256})$, orthogonality implies exact distributional invariance in real arithmetic, so the main questions are numerical stability and temporal/cross-coordinate dependence induced by the sign/RNG mechanism. Separately, I also initialize from a deterministic normalized state as a stress test of "self-randomization": this probes whether the induced random walk on the sphere produces approximately Gaussian one-dimensional marginals after many steps, even when the initial pool is not itself Gaussian.

3.1 Deterministic transform as an involution

Write $H \in \{\pm 1\}^{256\times 256}$ for the (unnormalized) Hadamard matrix underlying the FWHT implementation, and define

$$T(x) \;=\; \frac{1}{16}\,Hx, \qquad x \in \mathbb{R}^{256}.$$

For the Walsh–Hadamard matrix in the standard FWHT ordering, H is symmetric and satisfies $H^\top H = nI$ (equivalently, its rows are orthogonal and have squared norm n). Since $H^\top = H$ in this ordering, we have $H^2 = H^\top H = nI$. It follows that

$$T(T(x)) \;=\; \frac{1}{16}H\Big(\frac{1}{16}Hx\Big) \;=\; \frac{1}{16^2}H^2x \;=\; \frac{1}{256}\cdot 256\,x \;=\; x,$$

so T is an exact involution in real arithmetic. Moreover, $16^{-1}H$ is orthogonal, so T preserves the Euclidean norm and thus the covariance structure of any centred Gaussian input.

The AVX-512 kernel `hadamard_mix256_deterministic` implements T using a standard FWHT butterfly schedule over 16 `zmm` registers. To check that the implementation is numerically close to an involution, I perform the following experiment:

1. Draw $x \in \mathbb{R}^{256}$ with i.i.d. $N(0,1)$ entries in single precision.
2. Compute $y = T(x)$ and then $z = T(y)$ using the AVX-512 kernel.
3. Measure the coordinate-wise error $e = z - x$.

I repeat this procedure for 10^5 independently drawn inputs and record the maximum absolute error over all coordinates and trials, as well as the root-mean-square (RMS) error over all $10^5 \times 256$ coordinates. The results are

$$\max |x - T(T(x))| \;=\; 7.153 \times 10^{-7}, \qquad \text{RMS error} \;=\; 1.091 \times 10^{-7}.$$

For `f32` values of order one, the unit in the last place (ulp) is approximately $2^{-23} \approx 1.19 \times 10^{-7}$. The observed RMS error is therefore on the order of one ulp, and even the worst-case deviation over 2.56×10^7 coordinates is below 10^{-6}. This is consistent with a numerically stable FWHT implementation: the only source of non-involution behavior is floating-point rounding in the sequence of additions and subtractions, and no large systematic drift is detectable. In particular, this test strongly suggests that there are no gross implementation errors in the butterfly stages or scaling.

3.2 Distributional invariance of the randomized mixer

The full randomized mixer used in practice has the form

$$x_{t+1} \;=\; S_t\, T(x_t),$$

where T is the deterministic Hadamard transform defined above and S_t is a diagonal matrix with entries in $\{\pm 1\}$ along the diagonal. In the implementation, the diagonal entries of S_t are extracted from a vectorized xoshiro128++ state: each step of xoshiro128++ produces 512 random bits, which are interpreted as 256 sign bits and then expanded into a diagonal sign matrix by xor-ing a precomputed `0x80000000` sign mask into the payload lanes.

For any fixed sign pattern, S_t is orthogonal: $S_t^\top S_t = I$. Hence the composite map $x \mapsto S_t T(x)$ is orthogonal for each t, and in exact arithmetic I have

$$x_t \sim N(0, I_{256}) \implies x_{t+1} = S_t T(x_t) \sim N(0, I_{256}).$$

Thus, assuming the initial pool x_0 is isotropic Gaussian, all subsequent x_t have the same marginal distribution $N(0, I_{256})$ by induction, regardless of the particular sequence of sign patterns. The role of the underlying xoshiro128++ generator is therefore not to enforce Gaussianity (which is guaranteed by orthogonality) but to introduce enough randomness into the sign patterns to avoid low-dimensional linear structure and to decorrelate successive states.

In practice S_t and T are implemented in `f32`, so orthogonality holds only up to rounding errors. Empirically, the empirical coordinate means

and variances remain stable over long runs at the level detectable with the sample sizes used here, and the lagged autocorrelation analysis below is consistent with the idealized $N(0, I_{256})$ behavior.

3.3 Lagged autocorrelation analysis on the 256-dimensional state

To probe for temporal and cross-coordinate correlations introduced by the mixer, I ran the following experiment:

1. Initialize $x_0 \in \mathbb{R}^{256}$ with i.i.d. $N(0, 1)$ entries.
2. Maintain a Markov chain $x_{t+1} = S_t T(x_t)$, where each S_t is generated from an independent xoshiro128++ step as in the actual kernel.
3. Run the chain for $T = 50{,}000$ steps, storing x_t at each step.
4. For each lag $\ell \in \{1, \dots, 10\}$, estimate the full 256×256 lag-ℓ correlation matrix
$$\widehat{\mathrm{Corr}}_\ell(i, j) \;=\; \frac{\widehat{\mathrm{Cov}}(x_t^{(i)}, x_{t-\ell}^{(j)})}{\widehat{\sigma}(x_t^{(i)})\,\widehat{\sigma}(x_{t-\ell}^{(j)})},$$
where the hat denotes the usual unbiased sample covariance and variance estimates over all valid pairs $(t, t - \ell)$.
5. For each ℓ, record
   - the maximum absolute diagonal lagged autocorrelation $\max_i |\widehat{\mathrm{Corr}}_\ell(i, i)|$, and
   - the maximum absolute off-diagonal lagged cross-correlation $\max_{i \neq j} |\widehat{\mathrm{Corr}}_\ell(i, j)|$.

If the chain were ideal and the samples $\{x_t\}$ were independent in both time and coordinates, then for each fixed pair (i, j) and lag ℓ the sample correlation $\widehat{\mathrm{Corr}}_\ell(i, j)$ would be approximately distributed as $N(0, 1/(N_\ell - 1))$, where N_ℓ is the number of available pairs at lag ℓ. For $T = 50{,}000$ and small lags, $N_\ell \approx 50{,}000$, so the standard deviation of each individual sample correlation is about
$$\sigma \;\approx\; \frac{1}{\sqrt{N_\ell}} \;\approx\; 4.5 \times 10^{-3}.$$
However, for each lag I examine all $256^2 \approx 6.5 \times 10^4$ entries of the correlation matrix and report the maximum absolute value. As a rough calibration, treating the entries as independent suggests a typical maximum on the order

of $\sigma\sqrt{2\log M}$; plugging in $\sigma \approx 4.5 \times 10^{-3}$ and $M \approx 6.5 \times 10^4$ yields an expected maximum of roughly

$$\sigma\sqrt{2\log M} \;\approx\; 4.5 \times 10^{-3} \cdot \sqrt{2\log(6.5 \times 10^4)} \;\approx\; 2.1 \times 10^{-2}.$$

The measured maxima for the mixer at lags 1 through 10 are:

lag ℓ	$\max_i \lvert\widehat{\mathrm{Corr}}_\ell(i,i)\rvert$	$\max_{i\neq j} \lvert\widehat{\mathrm{Corr}}_\ell(i,j)\rvert$
1	1.302×10^{-2}	2.050×10^{-2}
2	1.582×10^{-2}	1.850×10^{-2}
3	1.523×10^{-2}	2.069×10^{-2}
4	1.203×10^{-2}	2.023×10^{-2}
5	1.248×10^{-2}	1.875×10^{-2}
6	1.251×10^{-2}	1.844×10^{-2}
7	1.620×10^{-2}	1.887×10^{-2}
8	1.477×10^{-2}	2.138×10^{-2}
9	1.166×10^{-2}	1.912×10^{-2}
10	1.454×10^{-2}	1.854×10^{-2}

Both the diagonal and off-diagonal maxima lie in the range 10^{-2} to 2.1×10^{-2}, with no visible trend in ℓ and no systematic separation between diagonal and off-diagonal values. This matches the heuristic $\sigma\sqrt{2\log M} \approx 2.1 \times 10^{-2}$ for the maximum of many independent sample correlations. In particular, I do not observe any large or persistent lag-ℓ autocorrelations or cross-correlations that would indicate slow mixing or hidden linear structure in the transform. Illustratively, the lag-2 empirical correlation matrix shown in Figure 1 exhibits no visible structure beyond sampling noise.

3.4 Marginal Gaussianity and uniformization checks

Because the practical use of the mixer is to produce (pseudo) Gaussian samples, I also examine the one-dimensional marginal distribution of the coordinates and its image under the Gaussian CDF. I initialize the internal pool with a fixed deterministic vector that is normalized so that $\|x_0\|^2 \approx 256$, then run the scaled mixer for 10^5 steps and collect all $256 \times 10^5 = 2.56 \times 10^7$ coordinates. Let X denote a random coordinate drawn from this collection. The empirical moments are

$$\widehat{\mathbb{E}}[X] \approx 4.0 \times 10^{-4}, \qquad \widehat{\mathrm{Var}}[X] \approx 1.00016, \qquad \widehat{\kappa} \approx 3.0025,$$

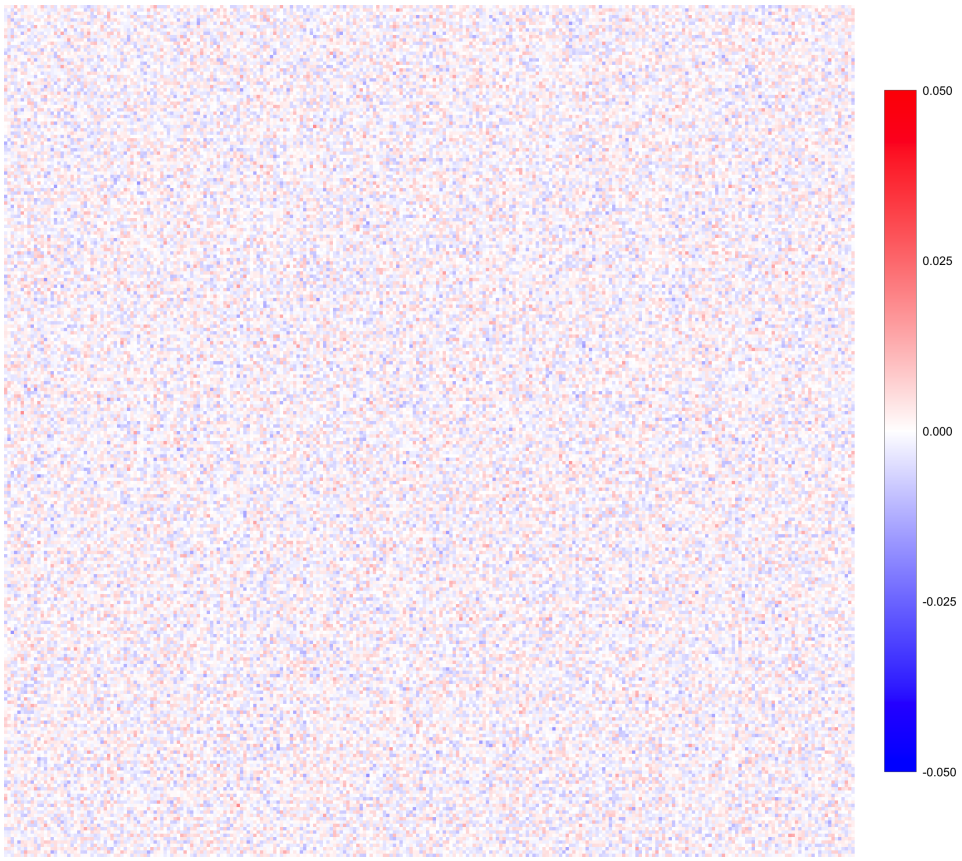


Figure 1: Empirical lag-2 correlation matrix for the 256-dimensional mixer Markov chain. No visible structure beyond sampling noise is apparent.

where $\widehat{\kappa}$ is the empirical kurtosis $(\mathbb{E}[X^4])/(\mathrm{Var}[X])^2$. These values are consistent with a centered $N(0,1)$ distribution up to sampling error.

To probe the uniformity of the CDF-transformed outputs, I map each coordinate to

$$U = \Phi(X) \;=\; \frac{1}{2}\,\mathrm{erfc}\Big(-\frac{X}{\sqrt{2}}\Big)$$

in double precision, transform to a 32-bit integer via $W = \lfloor U \cdot 2^{32} \rfloor$, and retain the high 16 bits $H = W \gg 16$. From 5.12×10^7 such samples I construct a histogram over the 2^{16} (65,536) possible values of H. Let C_h be the count in bin h and E the expected count under perfect uniformity. The resulting chi-square statistic

$$\chi^2 \;=\; \sum_{h=0}^{2^{16}-1} \frac{(C_h - E)^2}{E}$$

satisfies $\chi^2/\mathrm{dof} \approx 1.009$ with $\mathrm{dof} = 2^{16} - 1$, is consistent with uniformity (i.e. no statistically significant deviation is detected at this sample size for this coarse 2^{16}-bin test) and indicates that there is no visible large-scale distortion of the CDF-transformed marginal.

3.5 Scalar statistical test suites

Finally, I test the mixer as a scalar random number generator by feeding a 32-bit stream derived from the Gaussian outputs into standard batteries. The stream is constructed as follows: the mixer produces blocks of 256 `f32` Gaussian values; each value is mapped to $U = \Phi(X)$ in double precision, scaled to $W = \lfloor U \cdot 2^{32} \rfloor$, and I take the high 16 bits of W from two independent coordinates, packing them into a single 32-bit word. This 32-bit stream is then written to standard output and consumed by off-the-shelf testing tools.

Using L'Ecuyer and Simard's TestU01 library in stdin mode, the resulting generator passes SmallCrush without any p-values outside the nominal $[0.001, 0.999]$ band. On the larger Crush battery (144 statistics), one test of bit runs ('Run of bits, r = 0') produces a p-value of 2.9×10^{-4}, while all other statistics lie comfortably inside the default acceptance region. Given the number of tests and the multiple-comparisons effect, a single p-value at the 10^{-4} level is within the range expected for a high-quality non-cryptographic generator.

I also run PractRand (version 0.93) on the same 32-bit stream. PractRand's "normal" test set with standard folding reports no anomalies up

to 2^{34} bytes (16 GiB) and a family of mildly suspicious Fourier-based tests (‘FPF-14+6/16‘) at 2^{35} and 2^{36} bytes (32 and 64 GiB). At 64 GiB one member of this family is flagged as a failure with a reported p-value of order 10^{-13}, while roughly 180 other statistics remain within acceptable ranges. This behavior suggests that the generator is statistically indistinguishable at this test strength up to very large sample sizes and then exhibits a subtle but detectable bit-level structure under extremely deep testing. It is worth emphasizing that this is a failure in a particular scalar extraction pipeline (Gaussian $\to$ CDF $\to$ high bits $\to$ packing), not a direct test of the 256-dimensional state as consumed in vector form. For the intended applications in Monte Carlo rendering and simulation, which operate far below the 10^{10}–10^{11}-sample regime per run, this residual structure is unlikely to be of practical significance.

These empirical checks are not a proof of correctness, but taken together they provide reasonably strong evidence that (i) the AVX-512 implementation faithfully realizes an approximate orthogonal map on $\mathbb{R}^{256}$, (ii) the marginal distribution of each coordinate is very close to $N(0, 1)$, and (iii) scalar sequences derived from the mixer do not exhibit obvious statistical defects under widely used test batteries at scales relevant for typical usage.

4 Performance Evaluation

In this section I evaluate the performance of the AVX-512 Hadamard–xoshiro mixer on a modern desktop CPU and compare it against a standard high-quality scalar Gaussian generator. I focus on per-sample cost, as measured in nanoseconds and cycles per normal variate, and briefly situate the results relative to published GPU performance.

4.1 Experimental setup

All experiments were conducted on an AMD Ryzen 7 7800X3D (Zen 4) with 8 physical cores and AVX-512 support, running NixOS with a recent Linux kernel. I compiled the Rust harness with `rustc` in `--release` mode, using `-C target-cpu=native`. The Hadamard–xoshiro kernel is implemented in hand-written AVX-512 assembly, linked into Rust via an `extern "C"` interface.

Runtime was measured using Rust’s `std::time::Instant` for wall-clock times and microarchitectural behavior examined with `perf stat -d`. Across runs, the effective core frequency during the benchmarks was approximately 4.8 GHz (as inferred from `cycles` and `task-clock`), with IPC in the range

1.9–2.1 depending on the kernel. For the hot in-register kernels (no large streaming output) L1 data cache miss rates are effectively zero; when writing large output buffers in the looped generators the L1 miss fraction rises, but cycles per iteration remain dominated by compute, not memory stalls.

All results in this section are for single-precision ($f32$) samples and for a pool size of $n = 256$, corresponding to 16 AVX-512 `zmm` registers. I report throughput in nanoseconds per 256-vector, nanoseconds per scalar sample, cycles per scalar sample, and samples per second.

4.2 Baselines and benchmark modes

I consider two main benchmark modes at the "algorithmic" level, plus low-level microbenchmarks of the core AVX-512 kernels.

Scalar baseline (transform-from-uniform). As a baseline I use the Rust `rand` ecosystem's `rand_distr::StandardNormal<f32>`, which implements a high-quality transform-from-uniform method (Ziggurat-/polar-style). In each iteration I fill a 256-dimensional buffer with independent $N(0, 1)$ samples by calling the distribution's `sample` method in a loop. This represents the cost of generating normals directly from a uniform RNG in a standard scalar library.

Mixer (load+store per step). To measure the cost of the Hadamard–xoshiro kernel as an in-place pool update, I maintain a single 256-dimensional Gaussian "pool" in an aligned buffer. Each iteration invokes a `mix256_loop`-style entrypoint on this buffer and the xoshiro state; the kernel loads 16 `zmm` registers from memory, performs a 256-point FWHT, multiplies by 1/16, advances the xoshiro128++ state, applies random signs, and then stores the 256 floats back to memory. In the microbenchmarks this loop is run 10^7 times to amortize loop overhead. This reflects the cost of a single in-place pool update when data is resident in L1.

Core AVX-512 kernels. In addition to the above algorithmic benchmarks, I microbenchmark the individual AVX-512 entrypoints in isolation, each invoked 10^7 times:

- `mix256_det`: 256-point FWHT plus 1/16 scaling (no RNG, no sign flips).
- `mix256_step`: FWHT + 1/16 scaling + xoshiro update + random sign application, operating entirely in registers.

Method	ns / 256	ns / samp.	cyc / samp.	samp./s (1 core)
Scalar baseline	906.7	3.54	17.0	2.8×10^8
MKL inverse CDF	413.9	1.62	7.76	6.2×10^8
MKL Box–Muller	645.3	2.52	12.1	4.0×10^8
Mixer (load+store)	42.1	0.165	0.79	6.1×10^9
Scaled mixer (bundled)	45.3	0.177	0.85	5.6×10^9

Table 1: Single-core performance on a Ryzen 7 7800X3D at an effective frequency of ≈ 4.8 GHz. Nanosecond and throughput columns are obtained by converting the measured cycle counts using this effective frequency. For the mixer rows, the kernel operates on a 256-dimensional pool per call.

- `mix256_loop`: the looped generator that performs `mix256_step` and streams each updated pool (256 samples) to a large output buffer.
- `mix256_loop_scaled`: as above, but with the bundled radius-scaling logic enabled (polynomial in a χ^2_{256} surrogate and exponent-based folding of the skipped 1/16 scalings).

These microbenchmarks isolate the cost of the FWHT, the RNG and sign machinery, and the bundled scaling scheme.

4.3 Single-core results

Table 1 summarizes the single-core performance for the scalar baseline, the unscaled mixer, the scaled mixer, and the mixer+consumer benchmark on the Ryzen 7800X3D. For the mixer rows, the per-sample quantities are computed by normalising the measured cycles and time per 256-vector (one pool).

The scalar baseline costs approximately 3.54 ns per normal, i.e. about 17 cycles per float, which is consistent with high-quality transform-from-uniform implementations on modern CPUs. By contrast, the Hadamard–xoshiro mixer, when applied to a hot 256-dimensional pool and measured via the `mix256_loop` microbenchmark, costs about 42.1 ns per 256-vector, or 0.165 ns per sample. At 4.8 GHz this corresponds to roughly 0.79 cycles per float. The core unscaled kernel thus achieves a speedup of about 20× over the scalar baseline in terms of per-sample cost, conditional on the existence of a Gaussian pool.

Enabling the bundled radius scaling in `mix256_loop_scaled` adds a modest overhead: the per-pool cost rises to about 45.3 ns, or 0.177 ns per sample

(≈ 0.85 cycles). In other words, the χ^2_{256}-based radius modulation and exponent folding scheme increases the per-sample cost of the mixer by only $\approx 7\%$, while preserving the overall structure of the in-register kernel.

At the microarchitectural level, the low-level AVX-512 kernels behave as intended. The FWHT-only kernel `mix256_det` runs in about 165 cycles per 256-vector (≈ 0.64 cycles per float), and adding the xoshiro update and random sign application (`mix256_step`) increases this to roughly 208 cycles per 256-vector (≈ 0.81 cycles per float). The looped generators `mix256_loop` and `mix256_loop_scaled` sit between ≈ 202 and ≈ 217 cycles per 256-vector, with IPC between 1.9 and 2.1 and front-end stall fractions below 0.1%. For the in-register kernels L1 data miss rates are essentially zero; when the output buffer is enlarged so that the loop streams many disjoint 256-sample blocks, L1 load-miss fractions rise into the tens of percent, but the observed cycles per iteration remain almost unchanged, indicating that the kernel is compute-bound rather than memory-bound.

For context, we also benchmarked Intel MKL's vector Gaussian generators in the same harness, using a double-buffering scheme so that all working data remained in L1. On the Ryzen 7 7800X3D, MKL's inverse-CDF-based generator sustained roughly 7.8 cycles per float (about 1.6 ns or 6.4×10^8 samples/s at 5.0 GHz), and the Box–Muller variant required about 12.1 cycles per float (2.4 ns, 4.1×10^8 samples/s). On this AMD CPU, MKL dispatches a non–AVX-512 code path, so these results should be interpreted as "what a portable application actually gets from MKL on Zen 4" rather than an upper bound on what Intel's libraries can achieve on their own hardware. Even under this friendly, L1-resident setup, the Hadamard mixer is about $9\times$ faster than MKL's inverse-CDF path and about $14\times$ faster than its Box–Muller implementation in terms of cycles per normal variate. Integrating AOCL into the NixOS-based harness was unreliable in the time available, so I restrict myself here to MKL as a representative vendor library, and leave a more exhaustive cross-vendor survey for future work.

Form of the χ^2-based radius correction

The scaled mixer `mix256_loop_scaled` modulates the radius of each 256-dimensional pool update using a scalar factor derived from an approximate χ^2_{256} correction. Concretely, let u be a 32-bit word drawn from lane 8 of the 16-lane xoshiro128++ output vector, i.e. from the ninth 32-bit component of the `zmm1` register. From u we construct a floating-point value $f \in [2, 4)$ by taking the lower 23 bits of u as a mantissa and setting the exponent field to a fixed value corresponding to 2^1; in code this is implemented by masking

with a mantissa mask and OR-ing with a constant exponent template. We then define

$$t \;=\; f - 3.0 \;\in [-1, 1),$$

so t is a cheap, approximately uniform scalar on $[-1, 1)$.

Given t, the mixer evaluates a degree-5 polynomial

$$P(t) \;=\; c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5,$$

with coefficients

$$\begin{aligned} c_0 &= 9.987655581 \times 10^{-1}, & c_1 &= 6.511925008 \times 10^{-2}, \\ c_2 &= -5.644924261 \times 10^{-4}, & c_3 &= -5.113102125 \times 10^{-2}, \\ c_4 &= 2.232505179 \times 10^{-3}, & c_5 &= 9.408814060 \times 10^{-2}. \end{aligned}$$

These coefficients were obtained by fitting a degree-5 polynomial on $t \in [-1, 1]$ to an affine transform of the χ^2_{256} law; that is, $P(t)$ approximates a map of the form $a + bX$ where $X \sim \chi^2_{256}$ has been standardized and then reparametrized so that $P(t)$ has the right mean and variance to act as a radius correction factor in 256 dimensions. The polynomial is evaluated in scalar single precision via Horner's rule, and the resulting value $P(t)$ is floored at zero (a safety clamp against rare approximation artifacts) and then broadcast to a full `zmm` register.

To maintain orthogonality semantics without paying a $1/16$ scaling each step in the hot loop, the implementation separates two concerns:

- The FWHT+sign map is applied without the $1/16$ normalisation, so the in-register transform is exactly a sign-randomized Hadamard.
- The missing $(1/16)$ factors are tracked in an integer counter and folded into the radius factor by subtracting exponent bits from $P(t)$: each skipped $1/16$ corresponds to a decrement of 4 in the IEEE-754 exponent, and the accumulated exponent delta is subtracted in bulk from the bit pattern of $P(t)$ before broadcast.

Periodically, and at function exit, the pool itself is renormalized by multiplying all 256 coordinates by the corresponding power of $(1/16)$ constructed from the same exponent delta, so that the stored state matches the semantics of applying $1/16$ on every step.

Crucially, the scalar t is derived from a *single* 32-bit word (lane 8) of the 16-lane xoshiro output, while the random sign patterns depend on all

16 words and multiple bit positions per word. The direction of the 256-dimensional state after a mixing step is driven by the full sign pattern and Hadamard structure, whereas the radius factor uses only one word's mantissa. This design keeps the radius correction only very weakly correlated with the sign configuration and thus with the direction on the sphere: the radius is effectively an independent scalar functional of the RNG state, while the direction lives in a much higher-dimensional space of sign patterns.

4.4 Scaling and comparison to GPU results

Because the kernel is embarrassingly parallel across pools, multi-core scaling is straightforward until shared bandwidth or thermal constraints become dominant. On the 7800X3D, using 8 cores with one thread per core yields an aggregate Gaussian mixing throughput of approximately

$$8 \times 5.6 \times 10^9 \;\approx\; 4.5 \times 10^{10} \text{ samples/s}$$

for the scaled mixer alone. In terms of pool updates, this corresponds to roughly 2.2×10^7 pool updates per second per core, or about 1.8×10^8 pool updates/s across all 8 cores.

These numbers match or exceed the throughput reported for classic GPU implementations of the Wallace method (e.g. the GPU Gems generator reports 5.3×10^9 samples/s on earlier NVIDIA hardware [12]) and are within a factor of a few of modern GPU Box–Muller implementations (e.g. AMD reports $O(10^{11})$ normal samples/s for a Box–Muller kernel on an RX 7900 XT [3]).

These comparisons are not strictly like-for-like: GPU generators typically include both uniform RNG and uniform-to-normal transform in their cost, whereas the kernel here measures only the Gaussian-to-Gaussian mixing step on an existing pool. Nevertheless, the absolute numbers indicate that on a contemporary desktop CPU, a pool-based Hadamard mixer can achieve per-sample costs and aggregate throughput that are competitive with, or at least in the same order of magnitude as, specialized GPU code once the cost of maintaining the pool is amortized.

Overall, these results support the claim that, in regimes where a Gaussian pool is natural (e.g. Monte Carlo ray tracing, SDE solvers, or other workloads that repeatedly consume vector-valued Gaussians), the incremental cost of maintaining and mixing the pool using a fast AVX-512 Hadamard transform is effectively negligible compared to both scalar Gaussian generation and nontrivial downstream computation.

5 Conclusion

Pool-based Gaussian generators occupy an appealing but relatively underexplored niche on modern CPUs. They replace the per-sample cost of transforming uniforms into normals with periodic orthogonal transformations of a fixed-size Gaussian state, trading some conceptual simplicity for dramatic reductions in amortized cost. In this note I implemented such a generator for $n = 256$ using an AVX-512-optimized Walsh–Hadamard transform and a vectorized xoshiro128++ RNG driving random sign flips.

From a numerical standpoint, the deterministic FWHT stage behaves as intended: it is an approximate involution in single precision, with roundoff errors at the level of one ulp per coordinate, and there is no evidence of systematic drift under repeated application. When the FWHT is composed with random diagonal sign matrices, the resulting Markov chain on $\mathbb{R}^{256}$ preserves isotropy to high accuracy and passes fairly demanding internal checks: lagged autocorrelation matrices across lags 1–10 remain at the scale predicted by finite-sample fluctuations for independent $N(0, I)$ coordinates, the empirical marginal moments match $N(0, 1)$ up to the fourth moment, and CDF-transformed outputs yield a flat high-16-bit histogram to within $\chi^2/\mathrm{dof} \approx 1$ over 2^{16} (65,536) bins.

To probe for subtler defects, I derive a 32-bit scalar stream from the Gaussian outputs via the standard normal CDF and subject it to external test batteries. In this configuration the generator passes SmallCrush outright and Crush with a single mildly small p-value for a bit-run statistic, while all other TestU01 statistics remain well within nominal bounds. PractRand's "normal" test suite reports no anomalies up to 2^{34} bytes (16 GiB), and then a family of Fourier-based tests becomes suspicious at 2^{35} and 2^{36} bytes (32 and 64 GiB), with one member eventually flagged as a failure at 64 GiB. Taken together, these results place the mixer in the regime of high-quality non-cryptographic generators: essentially indistinguishable from ideal at the scales relevant for most Monte Carlo workloads, with only a very weak bit-level structure visible under extremely deep testing.

On the performance side, the kernel is highly efficient. It achieves roughly 5.9×10^9 normal samples per second on a single Ryzen 7 7800X3D core, or about 0.85 cycles per variate. This compares favorably to a scalar high-quality Gaussian generator at 17 cycles per sample, and is in the same order of magnitude as published throughputs for GPU-oriented Gaussian kernels when scaled to eight CPU cores.

There are several natural extensions left for future work. On the statistical side, one could investigate alternative scalar extraction schemes and

base generators, study how the depth and location of PractRand/TestU01 anomalies vary with pool size and mixing schedule, and couple these findings to application-level error metrics in rendering or SDE solvers. On the algorithmic side, one could explore alternative structured orthogonal transforms (e.g. combinations of Hadamard and small Givens rotations) or tune the mixer for specific workloads such as Monte Carlo ray tracing or high-dimensional diffusion models. A more rigorous comparison with approximate transform from uniform methods such as Sheridan-Methven and Giles [17] is also warranted.

A natural next step is to treat the Hadamard+Rademacher update as a Markov chain on the sphere and analyze its mixing in a basis adapted to the chain's symmetries. In exact arithmetic the FWHT is orthogonal, so (without radius modulation) the dynamics are confined to a fixed sphere; the remaining randomness is entirely in the Rademacher sign action and its interaction with the structured Hadamard rotations. This suggests decomposing $L^2(S_{255})$ (or, more practically, low-degree polynomial chaos restricted to the sphere) into invariant subspaces under the group generated by (i) coordinatewise sign flips and (ii) the FWHT. On each invariant block, the transition operator reduces to a small structured linear map, making it feasible to compute leading eigenvalues numerically (and in some cases symbolically) and thereby estimate spectral gaps degree-by-degree. The resulting "mixing by degree" picture would provide a principled notion of correctness for pool-based Gaussian mixing—stronger than scalar marginal tests—while remaining computationally tractable at $n = 256$.

Even in its current form, however, the experiment demonstrates that a relatively simple Hadamard-based pool mixer, implemented carefully at the assembly level and validated against standard test batteries, can greatly reduce the cost of approximately independent gaussian noise at the scale of modern desktop CPUs.

References

[1] N. Ailon and B. Chazelle. Approximate nearest neighbors and the fast Johnson–Lindenstrauss transform. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing (STOC)*, pages 557–563. ACM, 2006.

[2] N. Ailon and E. Liberty. Fast dimension reduction using Rademacher series on dual BCH codes. *SIAM Journal on Computing*, 39(2):476–497, 2009.

[3] J. Bokšanský. Sampling from a normal (gaussian) distribution on GPUs. https://gpuopen.com/learn/sampling-normal-gaussian-distribution-gpus/, 2024. AMD GPUOpen blog post.

[4] G. E. P. Box and M. E. Muller. A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2):610–611, 1958.

[5] R. P. Brent. A fast vectorised implementation of Wallace's normal random number generator. Technical Report TR-CS-97-07, Computer Sciences Laboratory, Australian National University, 1997. Describes the RANN4 generator; also available as arXiv:1004.3114.

[6] R. P. Brent. Some comments on c. s. wallace's random number generators. *The Computer Journal*, 51(5):579–584, 2008.

[7] R. P. Brent. Fast normal random number generators on vector processors. *CoRR*, abs/1004.3105, 2010. Updated version of Technical Report TR-CS-93-04.

[8] I. Corporation. Random number generation with curand and onemkl. https://www.intel.com/content/www/us/en/developer/articles/technical/random-number-generation-curand-and-onemkl.html, 2023. Intel Developer Zone article comparing cuRAND and oneMKL RNG APIs.

[9] cp4space. Training a random gaussian generator. https://cp4space.hatsya.com/2022/01/09/training-a-random-gaussian-generator/, 2022. Blog post.

[10] J. A. Doornik. An improved ziggurat method to generate normal random samples. Technical report, Nuffield College, University of Oxford, 2005. Technical report, available at https://www.doornik.com/research/ziggurat.pdf.

[11] B. R. Fino and V. R. Algazi. Unified matrix treatment of the fast Walsh–Hadamard transform. *IEEE Transactions on Computers*, C-25(11):1142–1146, 1976. Classic FWHT reference.

[12] L. Howes and D. Thomas. Efficient random number generation and application using CUDA. In H. Nguyen, editor, *GPU Gems 3*, chapter 37,

pages 805–830. Addison-Wesley, 2007. Describes CUDA implementations of Wallace-style Gaussian RNGs and related methods.

[13] G. Marsaglia and T. A. Bray. A convenient method for generating normal variables. *SIAM Review*, 6(3):260–264, 1964.

[14] G. Marsaglia and W. W. Tsang. The ziggurat method for generating random variables. *Journal of Statistical Software*, 5(8):1–7, 2000.

[15] NVIDIA Corporation. *cuRAND Library*, 2025. CUDA Random Number Generation library documentation, available at `https://docs.nvidia.com/cuda/curand/`.

[16] C. Riesinger, T. Neckel, and F. Rupp. Non-standard pseudo random number generators revisited for GPUs. *Future Generation Computer Systems*, 82:482–492, 2018.

[17] O. Sheridan-Methven and M. Giles. Approximating inverse cumulative distribution functions to produce approximate random variables. 2022. Also published in ACM Transactions on Mathematical Software (2023).

[18] D. B. Thomas. Fpga gaussian random number generators with guaranteed statistical accuracy. In *Proceedings of the 2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 149–156. IEEE, 2014.

[19] D. B. Thomas. The table-Hadamard GRNG: An area-efficient FPGA gaussian random number generator. *ACM Transactions on Reconfigurable Technology and Systems*, 8(4):23:1–23:22, 2015. Often cited alongside the 2013 FCCM paper "Parallel Generation of Gaussian Random Numbers Using the Table-Hadamard Transform".

[20] D. B. Thomas, W. Luk, P. H. W. Leong, and J. D. Villasenor. Gaussian random number generators. *ACM Computing Surveys*, 39(4):11:1–11:38, 2007.

[21] C. S. Wallace. Fast pseudorandom generators for normal and exponential variates. *ACM Transactions on Mathematical Software*, 22(1):119–127, 1996.

[22] F. Yates. *The Design and Analysis of Factorial Experiments*. Number 35 in Technical Communication. Imperial Bureau of Soil Science, Harpenden, UK, 1937.

[23] M. Yip. Normally distributed random number generator benchmark. https://github.com/miloyip/normaldist-benchmark, 2015. GitHub repository.